TAGUIG CITY UNIVERSITY
General Santos Ave., Taguig, 1632 Metro Manila
COLLEGE OF INFORMATION & COMMUNICATION TECHNOLOGY
Academic Year 2026 – 2027

H.A.J.K. HIGH-RISE

HOTEL RESERVATION SYSTEM

Prepared By:
Cortal, Kirk Patrick S.
Dela Cruz, Jasper E.
Ferolino, Abegail T.
Galzote, John Dave J.
Olaes, Hans Norman T.

BSCS 2B

Submitted to:
Prof. Rommel Dorin

November 2026

EXECUTIVE SUMMARY

The project is titled H.A.J.K. High-Rise Hotel Reservation System. H.A.J.K. represents the initials associated with the project developers, while “High-Rise” reflects the multi-floor property concept represented by the group name and logo. Designed as a comprehensive desktop solution utilizing VB.NET (Windows Forms) and MySQL, the system streamlines front-desk operations, room management, guest profiles, and secure administrative reports.

The application aims to address the operational inefficiencies of traditional manual recordkeeping by providing occupancy metrics, automated calculations for the number of nights and applicable VAT, role-based access control, and database integrity features. Furthermore, the platform integrates granular administrative controls, supporting role-based permissions, flexible promotional discounting, and detailed financial transaction tracking. Through exception handling and input validation across its modules, the system is designed to improve reliability and handle common operational errors. This documentation comprehensively details the structural framework, system user interfaces, exhaustive test case validation results, and the project development timeline.

Table of Contents

EXECUTIVE SUMMARY II

LIST OF TABLES IV

LIST OF FIGURES IV

INTRODUCTION 1

PROJECT DESCRIPTION 1

OBJECTIVES 2

SCOPE AND LIMITATIONS 2

SYSTEM REQUIREMENTS 4

FUNCTIONAL REQUIREMENTS 4

NON-FUNCTIONAL REQUIREMENTS 5

SOFTWARE REQUIREMENTS 5

SYSTEM ANALYSIS AND DESIGN 6

INTENDED USERS AND ACCESS 6

ADMIN ROLE 6

RECEPTIONIST ROLE 8

DATABASE DESIGN AND SCHEMA 11

ENTITY-RELATIONSHIP DIAGRAM (ERD) 11

TABLE RELATIONSHIPS AND INTEGRITY RULES 12

SOFT-DELETE FUNCTIONALITY 12

SYSTEM IMPLEMENTATION 13

TECHNOLOGY STACK 13

PRICING, DISCOUNTS, AND BILLING RULES 13

TESTING AND EVALUATION 16

TYPES OF TESTING EXECUTED 16

TEST CASES AND EXECUTION RESULTS 17

SUMMARY OF TEST RESULTS 20

DEVELOPMENT TIMELINE 21

USER MANUAL 23

SYSTEM WORKFLOW 23

CONCLUSION 24

APPENDICES 25

APPENDIX A – SYSTEM USER INTERFACE 25

APPENDIX B – TEST CASE DOCUMENTATION 33

LIST OF TABLES

Table 1: Administrator Role Module Specifications 6
Table 2: Hotel Room Categories and Standard Pricing Matrix 7
Table 3: Administrator System Reset Option and Execution Details 8
Table 4: Receptionist Role Module Specifications 8
Table 5: Reservation Module Functional Requirements 9
Table 6: Add Reservation Form Input Fields and Data Structure 10
Table 7: Guest Management Specifications and Field Requirements 10
Table 8: Hotel Discount Rates and Statutory VAT Treatment 13
Table 9: Billing Computation Steps and Mathematical Formulas 14
Table 10: Unit Test Cases for Financial Calculations and Date Validation 17
Table 11: Integration Test Cases for System Modules and Database Interactions 18
Table 12: System Test Cases for Authentication Security, Staff Management, and Administrative Maintenance 19

LIST OF FIGURES

Figure 1: Official Logo and Visual Branding of the H.A.J.K. High-Rise Hotel 1
Figure 2: Entity-Relationship Diagram (ERD) of the H.A.J.K. Hotel Reservation System 11
Figure 3: Official Receipt/Guest Folio Layout 15
Figure 4: Login Interface for H.A.J.K. High-Rise Hotel Reservation System 25
Figure 5: Admin Dashboard Overview and Real-Time Occupancy Metrics 25
Figure 6: Room Management Module and Inventory Interface 26
Figure 7: Add Room Dialog Form for Database Configuration 26
Figure 8: Staff Account Management and Role Assignment 27
Figure 9: Financial Reports and Revenue Generation 27
Figure 10: Settings Module – Discount Rates Configuration Tab 28
Figure 11: Settings Module – Change Password Security Interface 28
Figure 12: Settings Module – System Reset and Configuration Maintenance 29
Figure 13: Receptionist Dashboard Overview and Daily Front-Desk Metrics 29
Figure 14: Reservation Management Module Interface 30
Figure 15: Add Reservation Form for New Guest Booking Entry 30
Figure 16: Guest Management Module Interface 31
Figure 17: Add Guest Registration Modal Dialog 31
Figure 18: Guest Stay History Form and Transaction Interface 32
Figure 19: Staff Account Settings – Password Modification Interface 32

Figure 20: Invalid Login Credentials Error Output 33
Figure 21: Deactivated Account Login Restriction 33
Figure 22: Missing Guest Name Fields in Reservation 34
Figure 23: Self-Deactivation Prevention 34
Figure 24: Admin Account Deletion Restriction 35
Figure 25: Selective Database Reset Execution Test 35
Figure 26: Reservation Process Validation Test 36
Figure 27: Data Export Functionality & CSV Generation Test Case 36
Figure 28: Invoice Generation Validation & Exception-Handling Test Case 37
Figure 29: Financial Report PDF Export Test Case 37
Figure 30: Staff Account Creation & Database Persistence 38
Figure 31: Account Soft Deletion & Database Persistence 38
Figure 32: Staff Account Recovery Workflow 39
Figure 33: Reservation Deletion Prevention 39
Figure 34: Automatic Nights Duration Count Calculation 40
Figure 35: Auto-Generated Reservation Identifier 40
Figure 36: Invalid Date Sequence Validation Test Case 40
Figure 37: Standard Reservation Calculation Test 41
Figure 38: Extra Adult Occupancy Calculation 41
Figure 39: Statutory Discount Application Test Case 42
Figure 40: Room Capacity Exceeded Validation Test 42
Figure 41: Lean Season Discount & VAT Computation 43

INTRODUCTION

This section provides an overview of the H.A.J.K. Hotel Reservation System, outlining its background, overall project description, general and specific objectives, and operational scope and limitations.

Project Description

The H.A.J.K. Hotel Reservation System is a desktop-based hotel reservation software designed to automate and centralize front-desk and back-office operations. Developed using Visual Basic .NET (VB.NET) and backed by a MySQL/MariaDB relational database, the platform provides role-based access control catering specifically to two user tiers: Administrators and s. The system manages room classifications and physical inventory states (*Available, Occupied, Out of Order, Under Maintenance*), handles guest registration and historical stay tracking, and processes bookings with automated room-rate and extra-person billing calculations. Additionally, it features flexible discount engines supporting statutory (PWD/Senior Citizen) VAT exemptions and seasonal promotions, built-in financial reporting, invoice/receipt generation, and an administrative system reset utility**.**

The acronym H.A.J.K. represents the initials of the project developers, while "High-Rise" embodied the hotel's multi-floor concept, as represented by the group's branding. The system also uses the tagline "Where every stay feels extraordinary," which represents the group's goal of providing a convenient and comfortable hotel reservation experience through a simple and organized system.



Figure 1: Official Logo and Visual Branding of the H.A.J.K. High-Rise Hotel

Objectives

The general objective of this system is to design and develop a localized hotel reservation desktop system using VB.NET Windows Forms and MySQL that supports room inventory, guest and reservation records, role-based access, billing and report generation.

Specific Objectives:

1. Provide secure user authentication and role-based access for Administrator and Front Desk personnel.
2. Implement comprehensive CRUD (Create, Read, Update, and Delete) operations for managing guest reservations, room allocations, and stay information.
3. Calculate reservation charges based on room rate, number of nights, extra-person fees, discounts, and applicable Value-Added Tax (VAT) rules.
4. Record payment transactions securely, and generate or preview official billing documents, receipts, and comprehensive financial reports.
5. Implement secure administrative controls for system configuration, user management, and maintenance operations, including password-protected data resets.
6. Apply the four pillars of Object-Oriented Programming (OOP) to establish a modular, secure, and maintainable software architecture across all system components.

Scope and Limitations

The H.A.J.K. High-Rise Hotel Reservation System is designed to support the hotel's administrative and front-desk operations by providing a centralized platform for managing guest information, room inventory, reservations, billing, and payment records. The system includes a login and authentication feature with role-based access for two types of users: the Administrator and the Receptionist. The Administrator is responsible for managing staff accounts, maintaining room inventory records, managing discounts, viewing daily, monthly, and annual financial reports, and performing authorized system maintenance. Meanwhile, the Receptionist is responsible for managing guest records, creating, searching, editing, and canceling reservations, checking room availability, assigning rooms, processing check-in and check-out transactions, recording payment details, and generating or previewing invoices. The system also includes a Settings module that allows administrators and front-desk staff to update their account information. Furthermore, it supports the computation of reservation charges based on room rates, length of stay, additional-person fees, applicable discounts, and value-added tax (VAT), following the hotel's defined billing policies.

Despite its intended functionality, the H.A.J.K. High-Rise Hotel Reservation System has certain limitations. Although the system provides room inventory management and room assignment functionalities, it does not include the interactive floor map feature as proposed in

the initial project proposal. Room information and availability are managed through the system's records and interfaces rather than through a graphical representation of the hotel's floors. Additionally, the application is designed primarily for internal hotel personnel and does not provide a public-facing online reservation platform for guests. It does not include automated payment gateway integration, SMS or email notifications, or connections to external hotel booking platforms. Payment transactions are limited to recording payment information, such as the payment amount, method, reference number, date, and status. Moreover, the documented test cases focus on the system's primary user interface workflows, core operational tasks, and essential business logic. Furthermore, due to time constraints within the development schedule, the system does not support detailed audit log tracking or user activity histories, as the database schema does not include tables or triggers designed for audit trail logging. The system's functionality is also dependent on the availability of the MySQL database and a compatible Windows environment. Lastly, the accuracy of billing computations depends on the correct configuration of room rates, additional-person fees, discount rules, and applicable VAT policies.

SYSTEM REQUIREMENTS

This section outlines the functional, non-functional and software requirements necessary to ensure the effective deployment and operational performance of the H.A.J.K. Hotel Reservation System.

Functional Requirements

1. The system shall allow an authorized user to log in using a registered account.
2. The system shall provide role-based access for Admin and Receptionist accounts.
3. The Admin shall be able to add, edit, and delete or deactivate and reactivate staff accounts.
4. The Admin shall be able to add, edit, and delete room inventory records.
5. The system shall maintain room status as Available, Occupied, Out of Order, or Under Maintenance.
6. The system shall allow front-desk staff to create, search, edit, and cancel reservations.
7. The system shall generate reservation codes in the format YYYYMMDD-XXXX, based on booking date and daily counter.
8. The system shall allow reservation filtering by Pending, Check-In, Check-Out, No-Show, and Canceled status.
9. The system shall store guest name, contact details, valid ID type, and ID number.
10. The system shall calculate the number of nights from check-in and check-out dates.
11. The system shall calculate room charges, extra-person fees, discounts, VAT, and total amount due using the defined billing rules.
12. The system shall record payment amount, payment type/method, reference number, date, and status as supported by the database design.
13. The system shall generate or preview an invoice/receipt for a reservation.
14. The system shall allow reservation records to be exported to CSV/Excel.
15. The system shall allow receptionist to search guest records and view guest stay history.
16. The Admin shall be able to view daily, monthly, and annual financial reports and use the printable PDF feature.
17. The Admin shall be able to configure statutory and seasonal discounts.
18. The system shall provide account profile/password management.
19. The system shall provide an Admin-only reset function with password verification, and confirmation.
20. The system shall allow users to log out and return to the login screen.

Non-functional Requirements

1. Performance – Common actions such as opening records, searching, and saving should complete within a reasonable time on the intended computer and database setup.
2. Security – User passwords shall never be stored in plain text; they must be hashed using a secure one-way hashing algorithm before saving to the MySQL database. Additionally, deactivated accounts and soft-deleted records shall be restricted from active login while preserving historical database references.
3. Reliability – Deletion of operational records with active dependencies (e.g., reservations linked to payments) shall be blocked to maintain database integrity.
4. Usability – The user interface shall employ clear role-tailored dashboards, dynamic tab visibility, and standardized control enable/disable states based on user selections. Furthermore, system errors shall display user-friendly warning dialogs without crashing the application thread or exposing raw code to end-users.
5. Maintainability – Visual Basic .NET forms and controls shall maintain a clear separation between user interface controls, database access logic and repository helper classes. Moreover, the system shall run seamlessly on MySQL/MariaDB server instances with standard relational integrity rules enabled.

Software Requirements

1. Operating System – Windows version compatible with the selected .NET Framework/runtime.
2. Development and Runtime Environment – Visual Studio and associated .NET runtime tools utilized for building, debugging, and executing the desktop application.
3. Database Management System (DBMS) – MySQL Server or MariaDB (via XAMPP or standalone server instance) supporting relational constraints and InnoDB storage engine.

SYSTEM ANALYSIS AND DESIGN

This section details the structural architecture, user roles, workflow modules, and technical design specifications of the H.A.J.K. Hotel Reservation System. It establishes the system user hierarchy and defines functional responsibilities.

Intended Users and Access

This section outlines the primary user roles and their associated system privileges within the platform.

- Admin – Responsible for management of room inventory and staff accounts. The administrator possesses exclusive authority to configure hotel discounts, financial reports, and perform authorized system maintenance—including password-protected selective or factory data resets.
- Receptionist – Front-desk personnel are primarily responsible for managing day-to-day guest transactions, including creating and editing reservations, processing check-ins and check-outs operations, and billing workflows.

Admin Role

The Admin interface grants complete control over hotel setup, staff management, financial oversight, and system-wide configurations.

Table 1: Administrator Role Module Specifications

Menu / Module	Functions	Key Details
Home	Dashboard	Navigates to dashboard.
Room Management	Room Types	Defines room classifications such as room type name, room size, standard and maximum capacity, base rate, and extra adult fees at the database level
	Room Inventory	Keeps track of room inventory and allows administrators to add, edit, and delete rooms, displaying the room number, room type, status, room size, standard and maximum capacity, and the base rate.
Users Management	Staff Accounts	Create, edit, delete/deactivate staff accounts.
Reports	Financial Reports	Daily, monthly, and annual financial reports with printable PDF feature.
Settings	Discounts	Configure statutory and seasonal discounts.
	Account Management	Update account password and profile details for administrator account.
	System Reset	Password verification, confirmation, and summary message before reset.
Log Out	Session Exit	Returns to login screen.

1. Admin Dashboard

Displays real-time operational metrics and performance analytics, providing administrators with immediate visibility into:

- Available rooms – vacant room counter display
- Reservation
  - Arriving (expected guest arrivals for the day)
  - Departing (expected check outs for the day)
  - In house (currently checked in guests)
- Key performance
  - Daily revenue (real-time income totals)
  - Occupancy rate (percentage of total rooms occupied)

2. Room Management

2.1. Room Types

Defines standard room classifications, specifying dimensions, standard and maximum guest capacities, base nightly rates, and included amenities.

Table 2: Hotel Room Categories and Standard Pricing Matrix

Room Type	Size	Standard Capacity	Maximum Capacity	Base Rate / Night	Key Amenities & Features
Standard Room	22 m²	2 Adults	3 Persons	₱2,500.00	1 Queen Bed; Free Wi-Fi; Work Desk; Air Conditioning; Private Bathroom; Smart TV
Deluxe Room	32 m²	2 Adults	4 Persons	₱3,800.00	1 King Bed or 2 Twin Beds; Mini Fridge; Private Balcony; Coffee Maker; Work Desk
Executive Suite	48 m²	2 Adults	4 Persons	₱5,500.00	Separate Living Area; 1 King Bed; Kitchenette; Mini Bar; Bathtub; City View
Family Suite	55 m²	4 Adults	6 Persons	₱7,000.00	2 Queen Beds; Dining Table; Living Room Space; Extra Storage; 2 Smart TVs

2.2. Room Inventory States

Categorizes physical room operational conditions to ensure clear status tracking during booking, check-in, and maintenance.

- Available – vacant room that can be assigned to a reservation/check-in.

- Occupied – room currently assigned to an in-house guest.
- Out of Order – room unavailable for normal reservation use.
- Under Maintenance – room unavailable while maintenance is being performed.

3. System Reset

This admin-only function requires password verification, a confirmation step, and a summary message box.

Table 3: Administrator System Reset Option and Execution Details

Reset Type	Scope	Details
Factory Reset	Full reset	Wipes transaction history and guest records, and returns all rooms to Available.
Selective Reset	Selected data logs	Admin chooses what to purge.
	Guest history	Supports a date-range filter.
	Financial & billing transactions	Purges selected financial/billing transaction records.
	Room statuses only	Resets room statuses.

Receptionist Role

Defines the operational menu structure for front-desk personnel, focusing on day-to-day guest transactions, reservation processing, and room status inquiries.

Table 4: Receptionist Role Module Specifications

Menu / Module	Functions	Key Details
Home	Dashboard	Opens the dashboard and operational summaries.
Reservations	Reservation Records	Search/filter, add, edit, cancel, invoice, and export reservations.
	Add Reservation	Captures guest, stay, room, occupancy, discount, payment, and billing information.
Guest Management	Guest Records	Search by guest ID/name, edit profile, view stay history.
Settings	Account Management	Update account password and profile details
Log Out	Session Exit	Returns to login screen.

1. Receptionist Dashboard

Serves as the primary source of operational metrics for receptionists, presenting real-time counters for:

- Available rooms
- Reservation
    - Arriving (expected guest arrivals for the day)
    - Departing (expected check outs for the day)
    - In house (currently checked in guests)

2. Reservation Management

2.1. Reservation Records

Specifies the functional requirements for managing guest bookings, from searching active records to handling cancellations and generating official billing receipts.

Table 5: Reservation Module Functional Requirements

Area	Requirement
Search	Search by reservation code, guest name, or room number.
Status Filter	All, Pending, Check-In, Check-Out, No-Show, Canceled.
Reservation Code	Auto-formatted as YYYYMMDD-XXXX using booking date and daily counter.
Displayed Fields	Reservation code, guest name, status, room number, arrival date, departure date, nights, and total.
Add Button	Opens Add Reservation modal.
Edit Button	Modifies existing booking details.
Cancel Button	Marks reservation as canceled and makes the assigned room available.
Delete Button	Deletes selected record
Check-in button	Updates the selected reservation status to "Check-In" and sets the assigned room status to Occupied.
Check-out Button	Updates the selected reservation status to "Check-Out"
Process payment button	Opens the payment modal interface allowing users to record payment details.
Invoice Button	Generates/previews printable billing documents.
Export Button	Exports reservation records to CSV/Excel.

2.2. Add Reservation Form

Details the data input structure required to record guest profiles, stay details, room assignments, discounts, and payment methods during new bookings.

Table 6: Add Reservation Form Input Fields and Data Structure

Form Group	Fields / Controls
Guest Information	Surname, first name, middle name, contact, valid ID type, valid ID number.
Booking Information	Reservation code, check-in and check-out date, auto-calculated number of nights, assigned room number, adult count, child count.
Payment Information	Discount dropdown, payment method; payment reference, billing breakdown (Subtotal, discount amount, VAT, total amount)

3. Guest Management

Specifies the requirements for searching guest profiles, updating contact details, and reviewing comprehensive stay histories.

Table 7: Guest Management Specifications and Field Requirements

Area	Field	Requirement
Search	Guest ID / Name	Search guest records by guest ID or name.
Guest Record	Guest ID	Unique guest identifier in the database
	Guest Surname	Guest full name
	Guest First name	
	Guest middle name	
	Phone	Guest phone number.
	Email	Guest email.
	Total Visits	Number of recorded visits.
Actions/Buttons	Add	Adds new guest profile
	Delete	Deletes guest record
	Edit	Updates profile details.
	View Stay History	Pop-up showing reference number, room number, arrival date, departure date, and guest notes.

DATABASE DESIGN AND SCHEMA

This section shows the database design and relational structure of the H.A.J.K. Hotel Reservation System. It covers the organization of data stores in MySQL/MariaDB using primary keys, foreign keys, and database normalization strategies to avoid redundant data, keep records accurate, and support soft deletion of account records.

Entity-Relationship Diagram (ERD)

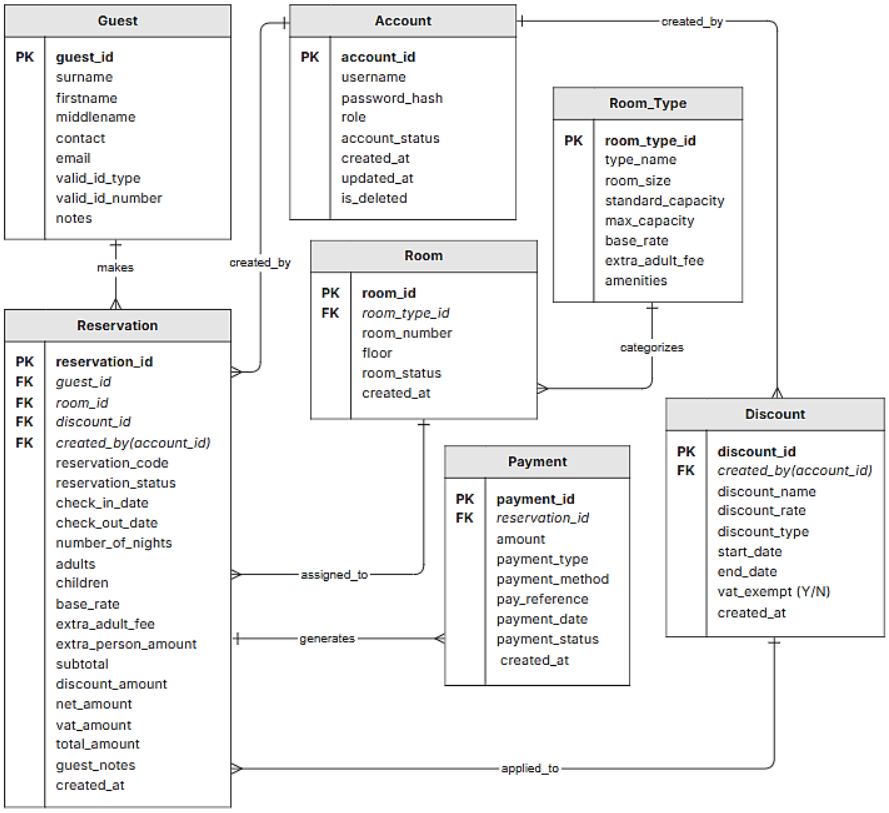


Figure 2: Entity-Relationship Diagram (ERD) of the H.A.J.K. Hotel Reservation System

The Entity-Relationship Diagram (ERD) above models the relational database structure for the system using **Crow's Foot notation**. It defines seven core entities, primary and foreign key constraints, and operational relationships to ensure referential integrity:

- Guest – Stores guest profiles (guest_id as PK) including identification details, contact info.
- Account – Manages system users (account_id as PK) and credentials for role-based authentication.
- Room_Type – Defines room classifications (room_type_id as PK), capacities, base rates, extra fees, and amenities.
- Room – Represents physical inventory (room_id as PK) linked to a Room_Type via room_type_id (FK).

- Discount – Contains statutory and seasonal rate rules (discount_id as PK) configured by an account (created_by FK).
- Reservation – Links guests, assigned rooms, accounts, and applied discounts (reservation_id as PK; guest_id, room_id, discount_id, created_by as FKs) while capturing stay durations and computed billing amounts.
- Payment – Tracks financial transactions (payment_id as PK) generated for specific bookings via reservation_id (FK).

Table Relationships and Integrity Rules

- Guests to Reservation – a single guest can make multiple reservations over time, but each reservation belongs to exactly one main guest profile.
- Reservation to Payment – a single reservation can generate multiple payment transaction records (e.g., down payments, deposits, or final balance settlements), with each payment record linking back to one specific reservation via reservation_id.
- Rooms to Reservation – a room can be booked across different date ranges, but each active reservation maps to one primary room assignment.
- Discount to Reservation – a defined discount rate or promotion can be applied to multiple guest reservations, while each reservation references a specific discount record via discount_id.
- Room Type to Room – a room type category (e.g., Standard, Deluxe, Suite) defines the rates and capacity for multiple individual physical rooms, but each room belongs to exactly one category.
- Account to Reservation – a single front-desk user account can process and create multiple guest reservations, with each reservation tracking who created it.
- Account to Discount – an admin user account can configure and create multiple discount options in the system, with each discount record linking to the administrative user via created_by (referencing account_id).

Soft-Delete Functionality

To protect historical transaction accuracy and support reporting without breaking database foreign key constraints, system records—including accounts, guests, and reservations—are deleted using boolean flags (is_deleted = TRUE). Queries throughout the application filter results using WHERE is_deleted = FALSE, allowing historical reservations and reports to access past guest and room data without exposing deleted entries in active management forms.

SYSTEM IMPLEMENTATION

This section outlines the software tools, database engine, and development environment used to build the desktop application, as well as the underlying operational rules governing room pricing, discount calculations, billing computations, and payment processing.

Technology stack

Highlights the software environment, framework, database engine, and connectors used to develop and execute the desktop application.

1. Visual Studio – Serves as the primary integrated development environment (IDE) used to write, compile, and debug the desktop application.
2. Visual Basic .NET – Provides the programming language used to develop the core business logic and handle user interface events.
3. Windows Forms – Supplies the framework for designing the desktop interface, interactive windows, and form controls.
4. MySQL Server – Handles persistent database storage for user accounts, room details, guest profiles, reservations, payments, and discounts.
5. MySql.Data – Functions as the ADO.NET connector enabling the VB.NET application to communicate with the MySQL database.
6. Git/GitHub – Used for version control, source code tracking, repository hosting, and team project management.

Pricing, Discounts, and Billing rules

1. Discount Rules

Outlines the percentage adjustments and tax treatment criteria for standard, statutory (PWD/Senior Citizen), and seasonal rate reductions applied to guest bookings.

Table 8: Hotel Discount Rates and Statutory VAT Treatment

Discount	Rate	Criteria	Calculation & Tax Rule
None	0%	Year-round standard booking	Standard calculation; 12% VAT fully computed and applied.
Statutory (PWD / Senior)	20% Off + VAT Exempt	Valid PWD or Senior Citizen ID holder	20% discount on Subtotal; skip 12% VAT computation.
Lean Season	15% Off Subtotal	Lean months	15% discount on Subtotal; standard 12% VAT computed on Net Amount.
Peak Season	10% Off Subtotal	Holidays & peak months	10% promotional discount on Subtotal; standard 12% VAT applies on Net Amount.

2. Billing Computation

Defines the sequential mathematical formulas used by the system to calculate room charges, extra guest fees, sub-totals, discounts, VAT, and final grand totals.

Table 9: Billing Computation Steps and Mathematical Formulas

Step	Formula / Rule
Room Charge	Base Rate × Nights
Extra Person Fee	Extra Adults × ₱800.00 × Nights
Subtotal	Room Charge + Extra Person Fee
Discount Amount	Subtotal × selected discount rate
Net Amount	Subtotal − Discount Amount
VAT	Net Amount × 0.12; skipped if Statutory discount applies
Total Amount Due	Net Amount + VAT

3. Payment Methods

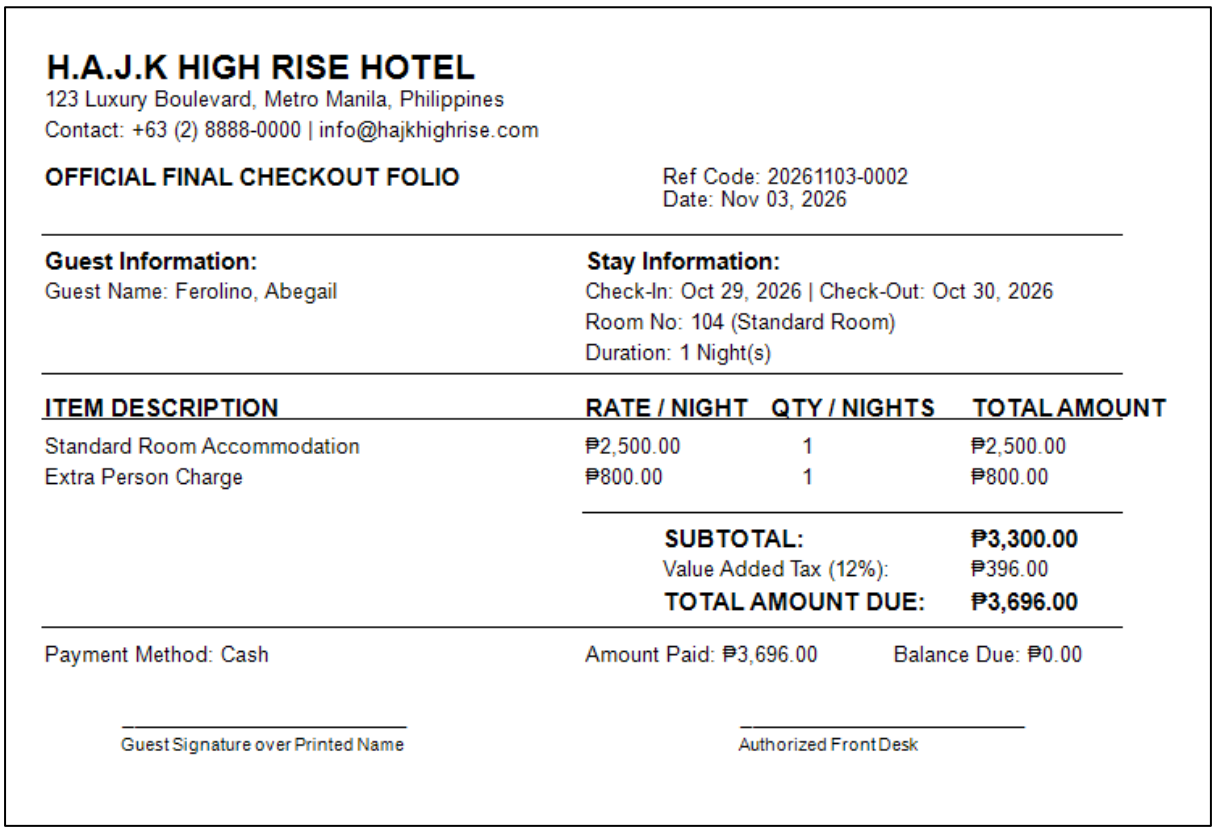
The system supports manual recording and verification of payments made through the listed payment methods. It does not directly connect to external payment gateways or automatically process digital transactions.

The supported payment options include:

- Cash – Handled directly at the front desk. The front-desk staff enters the cash amount received, and the system automatically computes any change due.
- GCash & Maya – Mobile wallet options for digital transactions. Receptionists verify the transfer on the payment terminal screen and enter the reference number into the reservation input fields.
- PayPal – Used for online deposits or advance settlements. A valid transaction ID is entered to link the electronic payment to the reservation.
- Debit/Credit Card – Processed using a physical terminal. The front desk staff records the terminal receipt reference number alongside the payment mode.
- Bank Transfer – Direct bank deposits or online transfers. Staff verify the guest's proof of payment before entering the transaction reference into the system.

4. Invoice Layout

The document is automatically compiled during the check-out transaction to provide guests with a complete and transparent financial record of their stay.



H.A.J.K HIGH RISE HOTEL
123 Luxury Boulevard, Metro Manila, Philippines
Contact: +63 (2) 8888-0000 | info@hajkhighrise.com

OFFICIAL FINAL CHECKOUT FOLIO

Ref Code: 20261103-0002
Date: Nov 03, 2026

Guest Information:
Guest Name: Ferolino, Abegail

Stay Information:
Check-In: Oct 29, 2026 | Check-Out: Oct 30, 2026
Room No: 104 (Standard Room)
Duration: 1 Night(s)

ITEM DESCRIPTION	RATE / NIGHT	QTY / NIGHTS	TOTAL AMOUNT
Standard Room Accommodation	₱2,500.00	1	₱2,500.00
Extra Person Charge	₱800.00	1	₱800.00

SUBTOTAL: **₱3,300.00**
Value Added Tax (12%): ₱396.00
TOTAL AMOUNT DUE: **₱3,696.00**

Payment Method: Cash
Amount Paid: ₱3,696.00
Balance Due: ₱0.00

Guest Signature over Printed Name

Authorized Front Desk

Figure 3: Official Receipt/Guest Folio Layout

Figure 3 presents the layout for the Official Final Checkout Folio generated during the checkout process to ensure full billing transparency. The document header displays essential metadata, including the hotel branding, contact information, unique transaction reference code, and issuance date. Beneath the header, guest personal details and stay parameters—such as check-in/check-out dates, room number, room category, and total night duration—are clearly structured. The central table shows item descriptions for individual charges, detailing nightly base rates, extra-person surcharges, and corresponding stay quantities to derive subtotal sums. The financial breakdown concludes with a 12% Value-Added Tax (VAT) computation, total amount due, recorded payment method, and remaining balance calculations. Finally, dual verification signature blocks are provided at the footer for both the guest and authorized front-desk personnel to validate and archive the settlement transaction.

TESTING AND EVALUATION

This phase was conducted to verify that the H.A.J.K. Hotel Reservation System operates in accordance with its design specifications, maintains computational accuracy, handles user input errors appropriately, and retains database integrity. The primary goal was to ensure that front-desk operations from initial guest registration and payment processing to report generation and invoice printing execute seamlessly without software runtime exceptions or financial computation errors.

Types of Testing Executed

To ensure full coverage across all architectural layers of the H.A.J.K. Hotel Reservation System, testing methodologies were categorized according to functional testing levels and codebase visibility types.

1. Unit Testing – Individual methods and low-level utility functions were validated in isolation. This included testing isolated VB.NET functions responsible for calculating stay durations, applying statutory 20% discount rates, and computing 12% Value-Added Tax (VAT).
2. Integration Testing - Verified that interconnected system modules communicated without data loss or execution errors. Integration tests focused on back-end connectivity between Visual Basic .NET forms and the MySQL database, verifying foreign key integrity across reservations, guests, and payments tables. Additionally, data integration was tested when exporting active records into external CSV files (opened in Excel) and PDF reports.
3. System Testing – Evaluated integrated modules and features such as login, reservation entry, account soft-deletion, system reset and logout. End-to-end workflows were executed, tracing guest check-in, room selection, bill calculation, soft-deletion status changes, and final receipt/folio generation.

Testing Approach:

1. Positive Testing – Conducted using valid data sets and expected operational flows to verify that system features perform according to requirements. Examples include processing valid room reservations, calculating accurate billing math with correct guest details, and generating official checkout folios.
2. Negative Testing – Evaluated using invalid, incomplete, or out-of-range inputs to verify that the application handles errors gracefully. Examples include attempting to enter a check-out date before the check-in date, leaving required fields blank, exceeding room capacity, and logging in with invalid credentials.

Test Cases and Execution Results

1. Unit Testing

This outlines the unit test cases executed to verify the accuracy of the system's core financial calculation and date validation logic. These tests validate individual back-end functions under both positive and negative scenarios, ensuring correct handling of stay durations, base rates, taxes, and boundary constraints.

Table 10: Unit Test Cases for Financial Calculations and Date Validation

Test Case ID	Target	Testing Approach	Scenario	Result
TC-01	Calculate Nights()	Positive Testing	Valid reservation date range: check-in (10/25/26) to checkout (10/30/26)	Correctly evaluated stay duration to 5 nights.
TC-02	Calculate Nights()	Negative Testing	Invalid date range: Checkout (10/30/26) prior to check-in (10/25/26)	Exception handled; displayed validation error dialog signal
TC-03	Calculate Total()	Positive Testing	Standard Calculation: BaseRate = ₱2,500.00, nights = 2, adults = 2 (StandardCapacity = 2), No Discount	Correctly calculated subtotal = ₱5,000.00, 12% VAT = ₱600.00, and total = ₱5,600.00
TC-04	Calculate Total()	Positive Testing	Extra Person Surcharge: BaseRate = ₱2,500.00, nights = 1, adults = 3 (StandardCapacity = 2)	subtotal computed as ₱3,300, 12% VAT as ₱396, and total as ₱3,696
TC-05	Calculate Total()	Positive Testing	Statutory Discount & VAT Exemption: subtotal = ₱5,000.00, 20% Senior/PWD discount, VATExempt = True	20% discount deducted (₱1,000.00); 12% VAT (₱0.00) skipped; total evaluated as ₱4,000.00
TC-06	Calculate Total()	Positive Testing	Seasonal Discount (Taxable): subtotal = ₱5,000.00, 15% Lean Season discount, VATExempt = False	Applied 15% discount; computed 12% VAT (₱510.00) on net amount; total = ₱4,760.00
TC-07	Calculate Total()	Negative Testing	Exceed Max Capacity; adults > selected room max capacity	Total textbox displays “Exceeds Max Capacity” and resets monetary textboxes to “0.00”
TC-08	Generate ReservationCode()	Positive Testing	Invoke code generation function with a specific booking date and counter.	Function correctly outputs a string matching the YYYYMMDD-XXXX format.

2. Integration Testing

This presents the integration test cases conducted to verify the seamless data exchange, cross-form navigation, and operational coherence between interdependent system modules—including parent form container switching, database foreign key synchronization, external file exports, and account management workflows.

Table 11: Integration Test Cases for System Modules and Database Interactions

Test Case ID	Target	Testing Approach	Scenario	Result
TC-09	Parent form navigation	Positive Testing	Click sidebar menu navigation buttons to switch views	Main panel switches content smoothly without opening separate pop-up windows.
TC-10	Reservation module to MySQL DB	Positive Testing	Submit valid reservation linking existing guest_id and room_id	Record saved across reservations and guests with maintained foreign key constraints.
TC-11	Reservation module to Excel/CSV	Positive Testing	Trigger "Export" action on active reservation list	Generated valid spreadsheet file cleanly with matched headers.
TC-12	Invoice Generation (PDF)	Negative Testing	Generate invoice without selecting an invoice type	The application displays the selection warning pop-up dialog
TC-13	Reports module to PDF	Positive Testing	Trigger “Export PDF” button	Generated valid PDF document cleanly with formatted tables and matching headers.
TC-14	Account Module to MySQL DB	Positive Testing	Create a new account with assigned role privileges	Account record is successfully written to the database
TC-15	Account module to MySQL DB	Positive Testing	Perform soft-delete action on a staff account	Record marked as deleted and hidden from active list while maintained in MySQL Database
TC-16	Account Restoration	Positive Testing	Select a deleted account from the "Show Deleted" view and trigger restoration	Account status updates to active, is_deleted resets to 0, and the record returns to the active staff list.
TC-17	Reservation Deletion Integrity	Negative Testing	Attempt to delete a reservation record that has active, linked payment transactions in the database.	System blocks the deletion operation and displays a warning pop-up dialog

3. System Testing

Shows the system-level test cases designed to evaluate security boundaries, role-based access control, input validation constraints, and database administrative maintenance tasks—including authentication checks for active and deactivated credentials, booking validations, self-deletion blocks, and system reset utilities.

Table 12: System Test Cases for Authentication Security, Staff Management, and Administrative Maintenance

Test Case ID	Target	Testing Approach	Scenario	Result
TC-18	Login authentication Module	Positive Testing	Log in with valid user account credentials (e.g., admin account)	Admin dashboard opens
TC-19	Login authentication Module	Negative Testing	Login with incorrect user credentials	Access is denied and a pop-up message displaying “Invalid username or password” appears.
TC-20	Login authentication Module	Negative Testing	Attempt to log into the system using the credentials of a deactivated/inactive account.	System blocks authentication, displays an error message indicating the account is inactive, and prevents access.
TC-21	Booking entry Validation	Negative Testing	Submit reservation form with missing mandatory fields (e.g., blank guest name, unselected room)	Submission blocked; visual input validation warnings displayed
TC-22	Staff Management Module	Negative Testing	Logged-in user triggers the self-account deactivation or deletion option.	System blocks action, preventing the logged-in user from deactivating or deleting their own account.
TC-23	Staff management Module	Negative Testing	On admin account, attempt to delete another admin account	System prevents admin account deletion and displays warning message.
TC-24	System Maintenance: System Reset	Positive Testing	Execute a selective reset (Guest History, 11/1/2026 to 11/2/2026) with admin password verification.	System requests confirmation, deletes only the selected records in the date range, and displays a summary of deleted reservations and guests.
TC-25	Session Management	Positive Testing	Click the "Logout" button from any active dashboard or management view.	Application displays confirmation dialog; when clicked ‘Yes’ it returns to the login screen

Summary of Test Results

A total of 25 test cases were executed across three hierarchical testing levels—Unit Testing (TC-01 to TC-08), Integration Testing (TC-09 to TC-17), and System Testing (TC-18 to TC-25)—to validate the functional correctness, database synchronization, security controls, and operational reliability of the H.A.J.K. Hotel Reservation System. All 25 test cases produced their expected results.

Unit testing successfully validated back-end computational functions (CalculateNights(), CalculateTotal(), and GenerateReservationCode()) and boundary constraints, confirming accurate calculations for stay durations, standard room rates, extra-person surcharges, statutory/seasonal discounts, 12% VAT, and automated reservation code formatting, while gracefully handling invalid date ranges and room capacity overloads. Integration testing assessed module interactions and database foreign key constraints, verifying smooth UI navigation, proper data synchronization between reservations and the MySQL database, clean external exports to Excel/CSV and PDF formats, reliable account creation, soft-delete and restoration workflows, and relational deletion safeguards that block deleting reservations with linked payment records. Finally, system testing examined security boundaries, input validation, role-based safeguards, administrative maintenance, and session management, confirming that the authentication module properly rejects invalid credentials and deactivated accounts, input validation catches missing booking fields, administrative safeguards prevent self-deletion and admin-on-admin account deletions, the selective system reset utility enforces password verification before purging specified historical records, and session termination via logout returns the user safely to the login screen. These results demonstrate that the application effectively handles boundary errors, protects operational data, and enforces strict database integrity standards.

DEVELOPMENT TIMELINE

This section outlines the project schedule and workflow phases followed during the planning, design, execution, and documentation of the H.A.J.K. Hotel Reservation System.

Week 1: Requirements Gathering and Scope Definition

During the initial week, the project team established the scope, main features, system limitations, and development collaboration tools. All team members attended an initial online video meeting on September 28, 2026 to discuss system requirements and project goals. The software was defined as an internal, front-desk desktop management system designed specifically for administrative and reception staff to handle daily hotel transactions. Boundaries were set to focus on a local desktop application built using Visual Basic .NET (VB.NET) and Windows Forms, explicitly ruling out public-facing web bookings, interactive graphical floor maps, and external SMS/email notification APIs. Additionally, operational constraints were documented, such as skipping complex audit log tables due to database scope limits. Development environments were configured in Visual Studio, and project repositories were set up on Git and GitHub repositories for easier team collaboration and code update tracking.

Week 2: Database Schema Creation & Business Logic Formulation

The second week focused on back-end database design, data integrity strategies, and formulating mathematical logic for hotel billing operations. Relational database tables for users, rooms, room types, guests, reservations, discounts, and payments were created in MySQL/MariaDB with explicit primary keys and foreign key constraints. To secure administrative access and protect system credentials, password hashing was integrated into the staff management logic so plain-text passwords are never stored directly in the database. Additionally, to protect historical records from permanent deletion during operational changes, a soft-deletion mechanism using boolean status flags (is_deleted) was integrated across key database tables, including account, guest, and reservation tables. Mathematical formulas were created to handle stay duration calculations, extra-person fees, subtotals, and the 12% Value-Added Tax (VAT). For statutory exemptions, such as Senior Citizen and PWD discounts, and promotional rates, logic was established to apply deductions to the subtotal (room charge plus extra-person fee) prior to tax computation. Lastly, database connectivity within the VB.NET environment was established using the MySql.Data ADO.NET provider connector.

Week 3: System UI Development & Forms Integration

The third week focused on developing the user interface in Visual Studio using a modular navigation design and connecting form controls to back-end SQL queries. Rather than opening separate pop-up windows for every module, a primary parent form was created

to house the main header and sidebar navigation menu. Custom User Controls were then developed for each screen content area, such as room management, guest registration, and reservation tracking, allowing smooth panel switching within the main window without form flicker or redundant loading. To improve front-desk monitoring, the dashboard UI was upgraded with real-time summary metrics—such as active reservations, available room counts, and total daily revenue—which automatically refresh using background timer controls. Furthermore, to simplify the registration process, supported payment method options were embedded directly into the Add Reservation form instead of requiring a separate pop-up window. Additionally, print layout preview options and report generation tools were integrated to allow front-desk operators to preview guest folios and export summary reports to CSV and Excel spreadsheet formats for financial bookkeeping.

Week 4: System Integration, Testing & Debugging

The fourth week was dedicated to complete integration testing, business logic verification, input validation, and bug resolution. Form-level validation rules were added to restrict invalid check-in date ranges, enforce required data fields, and trigger confirmation dialogs prior to record updates, soft-deletions, or application exits. The team tested full operational cycles covering guest bookings, payment records, automated report generation, and room checkouts. Financial calculations were verified across multiple test scenarios, including extra-person charges and Senior Citizen/PWD VAT exemptions, to guarantee exact balance and change computations without rounding errors. The phase concluded with fixing SQL query errors, resolving data type mismatches between VB.NET variables and MySQL fields, repairing spreadsheet export layouts, and adjusting UI element alignments.

Week 5: Final Documentation & Output Generation

The final week focused on compiling technical report chapters, refining visual documentation, and polishing final presentation deliverables. Throughout this phase, team members utilized remote messaging channels to coordinate document revisions, monitor project milestones, and finalize system updates in real time. Documentation sections were authored and integrated to cover the system design, database schema, technology stack specifications, scope boundaries, and implementation details. The layout for the generated Official Final Checkout Folio was finalized to display hotel header metadata, guest particulars, itemized billing records, subtotal and VAT breakdowns, and signature confirmation areas. Finally, test records were purged from the database, structured demonstration data was initialized, and the application was prepared for final evaluation.

USER MANUAL

This section provides the system's standard operating procedures, role-based workflows, and safety guidelines required for navigating and maintaining H.A.J.K. Hotel Reservation System effectively.

System workflow

1. **Logging In**
    - Launch the H.A.J.K. High-Rise Hotel Reservation System.
    - Enter the assigned username and password.
    - Select the Login button.
    - If credentials are valid and the account is active, the application opens the dashboard assigned to the account role. Otherwise, follow the displayed error message.
2. **Admin tasks**
    - Open the room management module to review and maintain room information.
    - Use staff account management to create, update, or deactivate staff accounts.
    - Open the reports module, choose the available report type or date range, and preview, print, or export the report.
    - Use settings to update supported password or discount options.
    - Use any system reset function only when authorized, after checking the selected reset scope and confirmation summary.
    - Log out when finished.
3. **Receptionist tasks**
    - Review the dashboard and room availability information.
    - Search for an existing guest or enter guest details as required.
    - Choose a room and enter check-in/check-out dates and guest counts.
    - Review the displayed billing breakdown and select the permitted discount/payment details.
    - Save the reservation and confirm that the generated reservation code or success message appears.
    - Use reservation search to review, edit, or cancel a booking according to permissions.
    - Select a reservation before requesting an invoice or receipt.
    - Log out when finished.

CONCLUSION

The H.A.J.K. Hotel Reservation System makes handling guest reservations, assigning rooms, and processing payments easier and smoother. Connecting it to a MySQL database means guest records and room statuses update in real time, which keeps all the data organized properly. On top of that, the app automatically handles all the mathematical logic for stay durations, extra guest fees, VAT, and discounts, saving time and cutting down on human error at check-in.

The system was developed using the four fundamental pillars of Object-Oriented Programming (OOP): encapsulation, inheritance, polymorphism, and abstraction. Encapsulation was applied to protect sensitive user credentials and restrict access to system functions based on user roles. Inheritance allows the reuse of base form layouts and structures, making the code easier to maintain. Polymorphism supports flexible handling of different pricing and discount calculations, while abstraction separates database operations from the user interface, keeping the application more organized.

The system underwent unit, integration, and system testing to evaluate its functionality and ability to handle different situations, including invalid inputs. Based on the documented test results, all 25 test cases produced their expected outcomes. Overall, the system achieved its main development objectives and provides a practical solution for organizing essential hotel reservation and front-desk operations. Although it still has limitations, it serves as a functional foundation that can be improved further to meet the needs of an actual hotel environment.

APPENDICES

Appendix A – System User Interface

This section presents the graphical user interfaces (GUIs) of the H.A.J.K. Hotel Reservation System, showcasing the administrative controls, management modules, and user workflows.

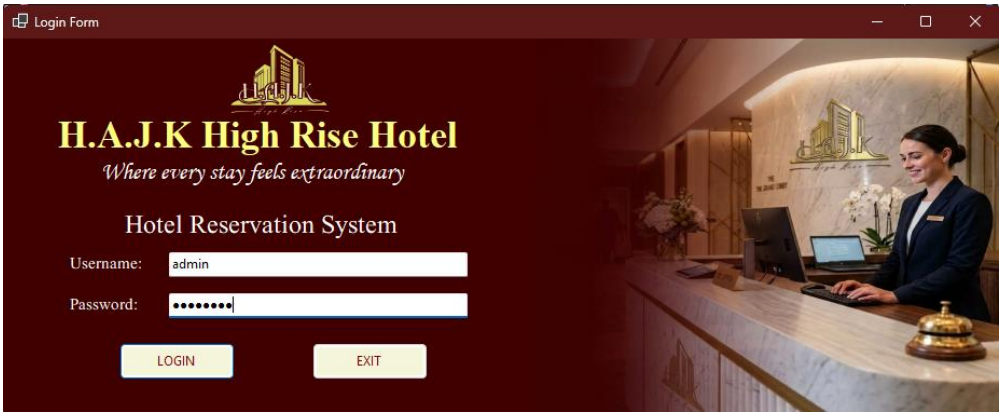


Figure 4: Login Interface for H.A.J.K. High-Rise Hotel Reservation System

The secure authentication gateway requiring administrative or staff credentials before granting system access.



Figure 5: Admin Dashboard Overview and Real-Time Occupancy Metrics

Displays live key performance indicators (KPIs) featuring dynamic metrics for available room count, occupancy percentage, daily and monthly revenue totals, and pending reservation counts (Arriving, In-house, Departing) updated dynamically upon system load.

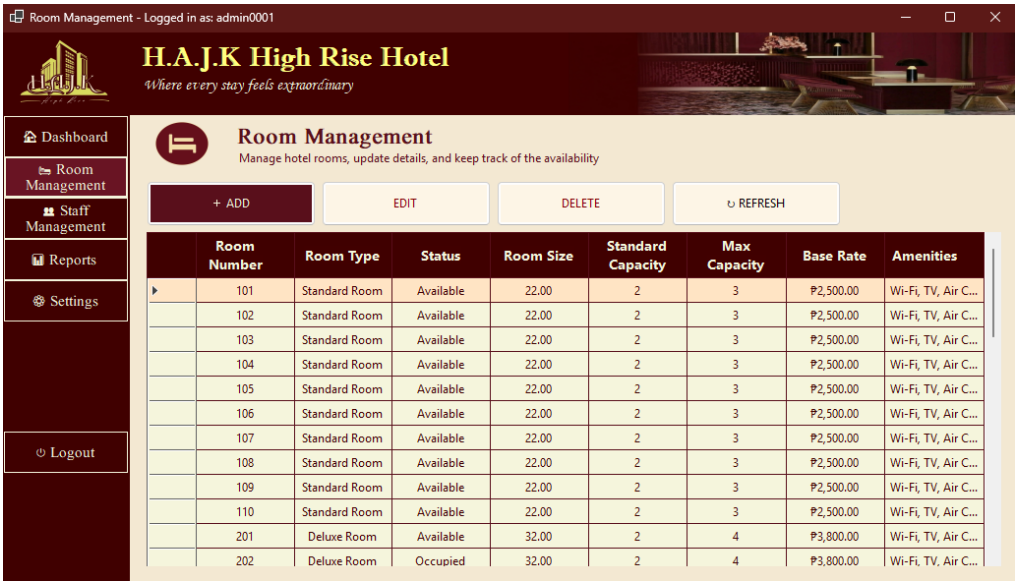


Figure 6: Room Management Module and Inventory Interface

The primary room inventory module providing comprehensive table views of room numbers, types, statuses, capacities, and base rates with CRUD action triggers.

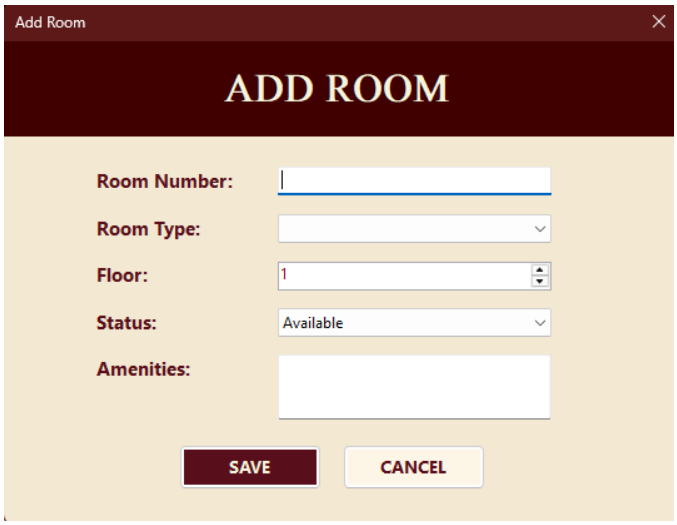


Figure 7: Add Room Dialog Form for Database Configuration

A modal used by administrators to add new room entries into the database, which includes field controls for room number specification, room type selection, floor level assignment, initial status setting, and amenity description listing.

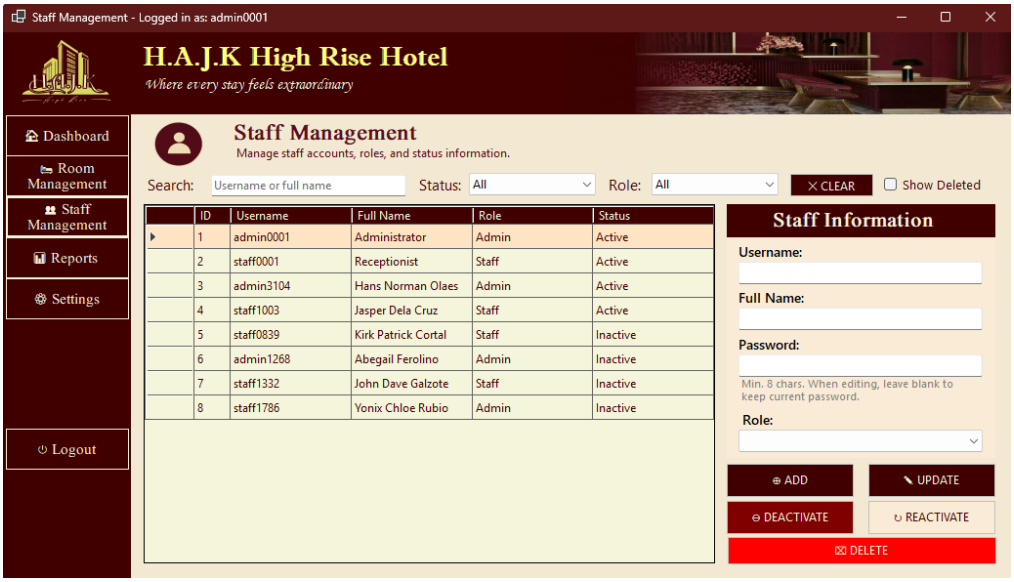


Figure 8: Staff Account Management and Role Assignment

The administrative staff control module for managing user accounts, access roles, and account statuses. It includes search filters by name or username, role filtering (Admin/Staff), account status toggles (Active/Inactive), checkbox for showing deleted accounts, and input fields to add, update, deactivate, reactivate, or delete user accounts.

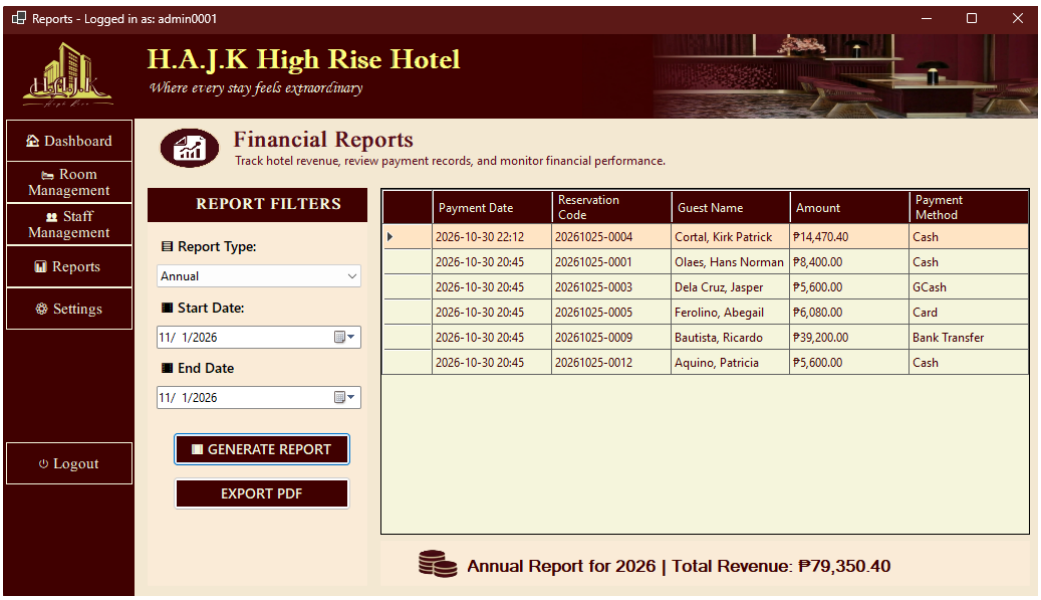


Figure 9: Financial Reports and Revenue Generation

The reporting interface used to compile and review historical revenue transactions. Users can set report criteria (Daily, Annual, Monthly, Custom Range), specify start and end dates, execute financial queries to generate itemized transaction tables, and export the generated ledger directly into a formatted PDF document.



Figure 10: Settings Module – Discount Rates Configuration Tab

The discount configuration interface displays discount types, percentage rates, and valid date ranges, allowing administrators to configure pricing rules applied during check-in or checkout billing processes.

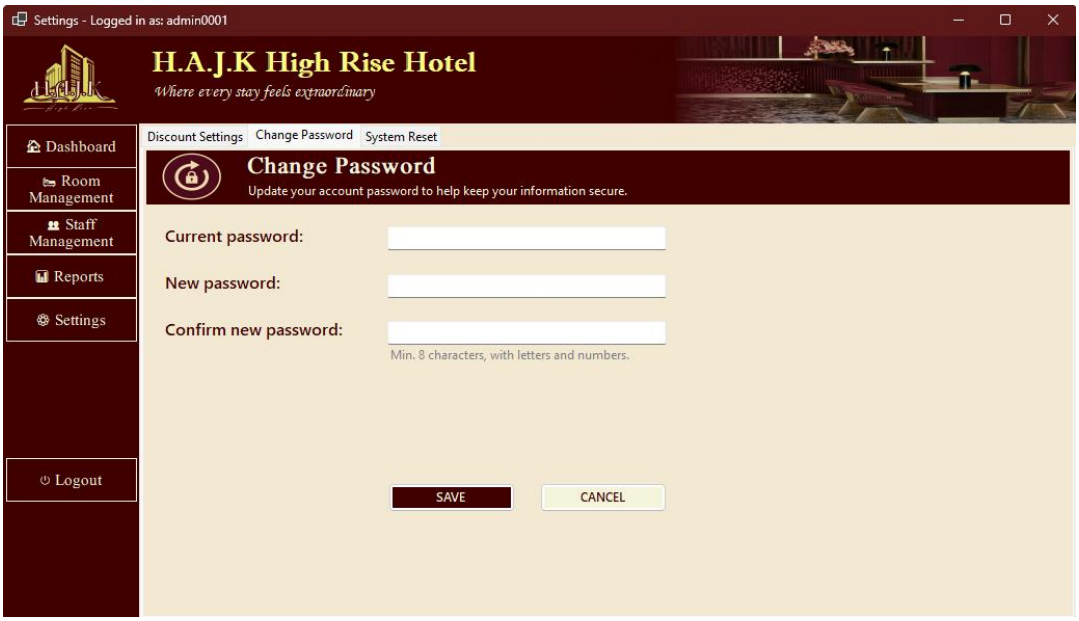


Figure 11: Settings Module – Change Password Security Interface

A dedicated account security panel allowing active users to update their login passwords. The form enforces password complexity policies (minimum 8 characters containing letters and numbers) and validates current credentials before committing hashed updates to the account table.

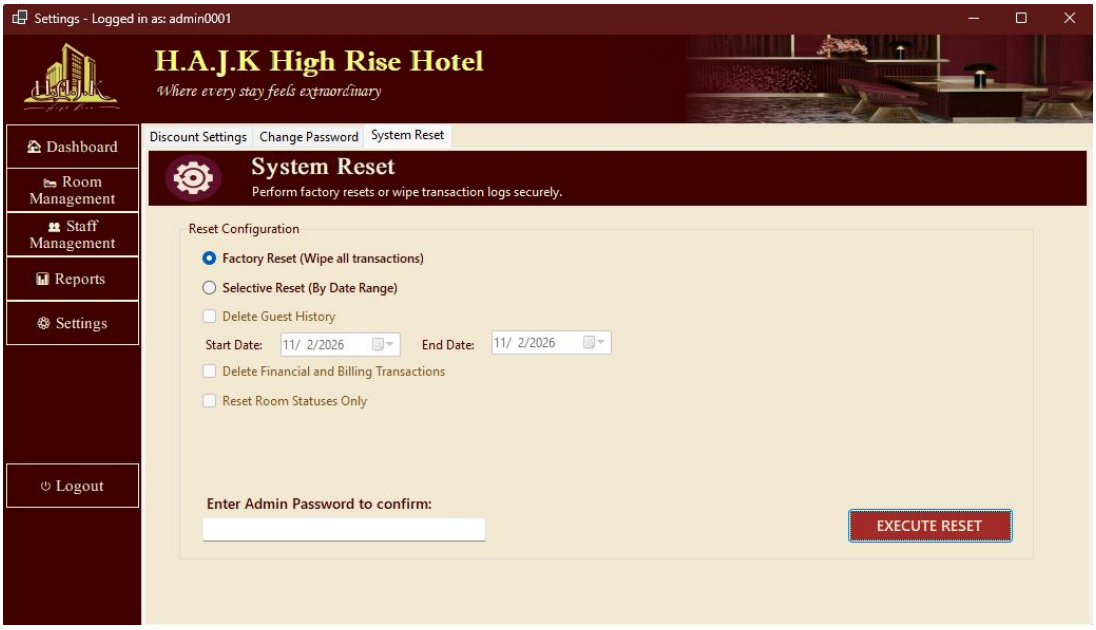


Figure 12: Settings Module – System Reset and Configuration Maintenance

An administrative maintenance utility providing factory reset and selective data purge capabilities. It allows administrators to wipe transaction logs, clear guest histories, reset billing records, or restore room status, secured via mandatory admin password re-verification.



Figure 13: Receptionist Dashboard Overview and Daily Front-Desk Metrics

The landing interface for front-desk staff or receptionist accounts upon successful authentication, which provides real-time tracking of summary metrics.

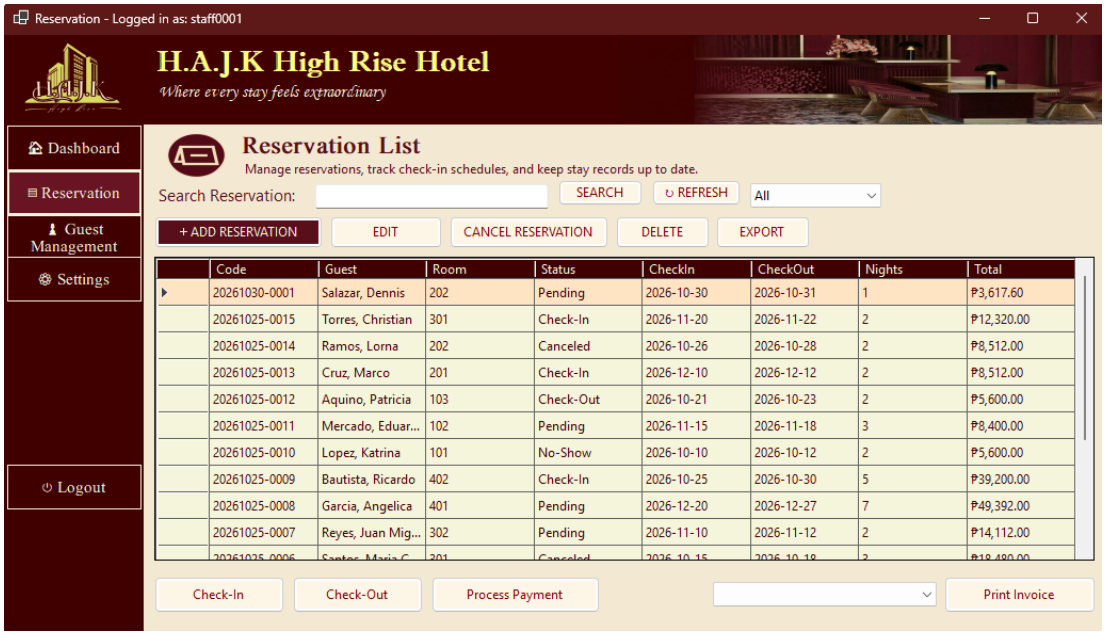


Figure 14: Reservation Management Module Interface

The comprehensive reservation list panel utilized by receptionists to manage booking schedules, track stay statuses (Pending, Check-In, Canceled, Check-Out, No-Show), and execute operational workflows such as processing check-ins, check-outs, handling payments, and printing invoices.

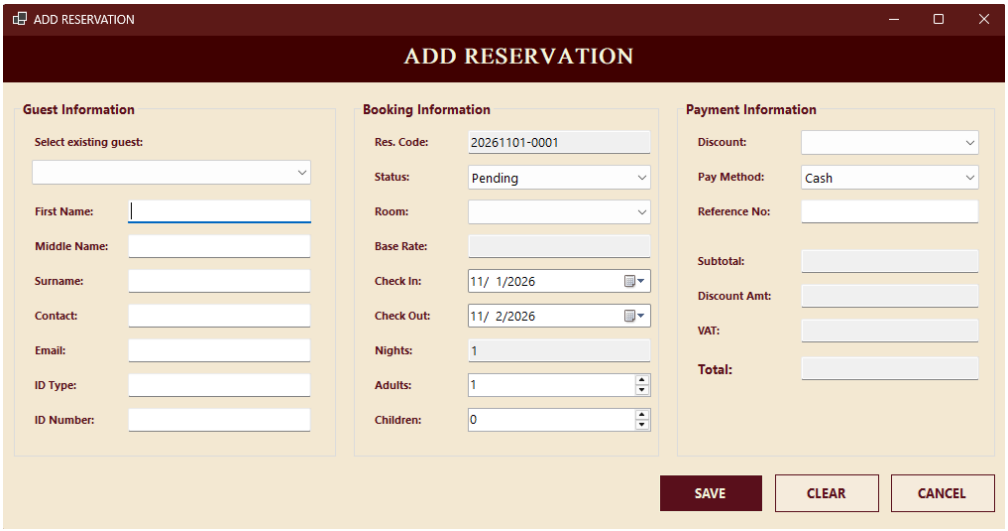


Figure 15: Add Reservation Form for New Guest Booking Entry

A detailed data-entry modal allowing staff to register new reservations. It organizes input fields into three logical sections: Guest Information, Booking Information, and Payment Information.

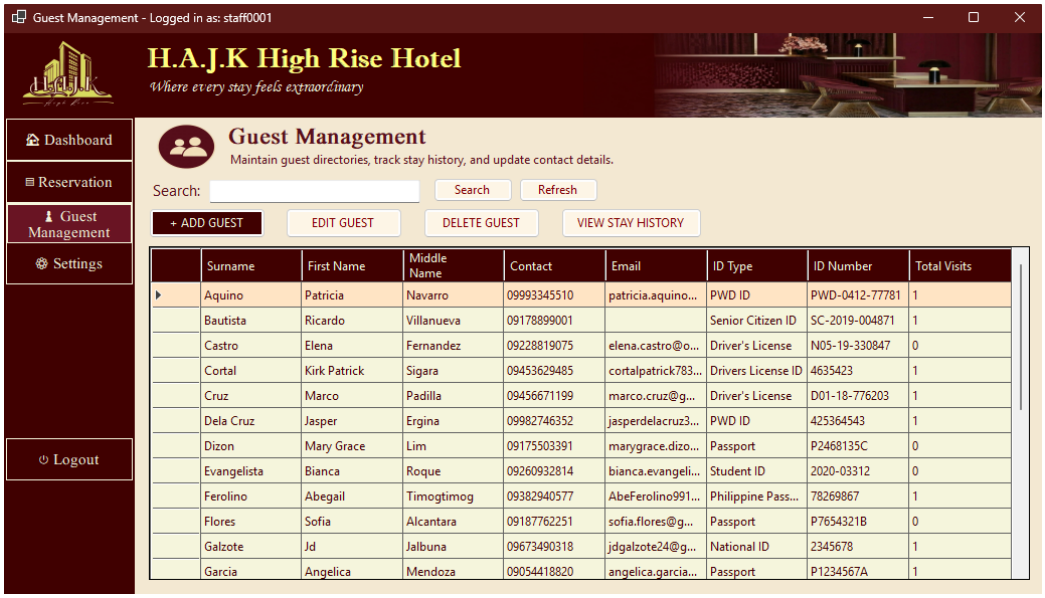


Figure 16: Guest Management Module Interface

The guest directory management enabling receptionists to maintain guest records, search directories by name or contact details, inspect valid identification types and numbers, and track historical visit counts (Total Visits) for returning patrons.

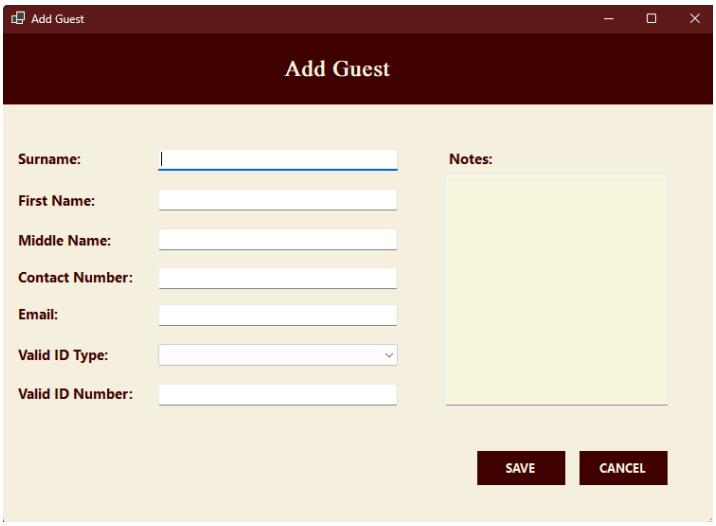


Figure 17: Add Guest Registration Modal Dialog

A dedicated form window used to capture and store new guest details into the database. It features fields for surname, first name, middle name, contact numbers, email addresses, valid ID specifications, and a dedicated notes section for special guest requests.



Figure 18: Guest Stay History Form and Transaction Interface

A modal view displaying the complete historical stay records for a selected guest. The interface features an itemized data grid tracking past and upcoming reservations by Reservation Code, Room Number, Check-In and Check-Out dates, stay Status, and Total Amount.

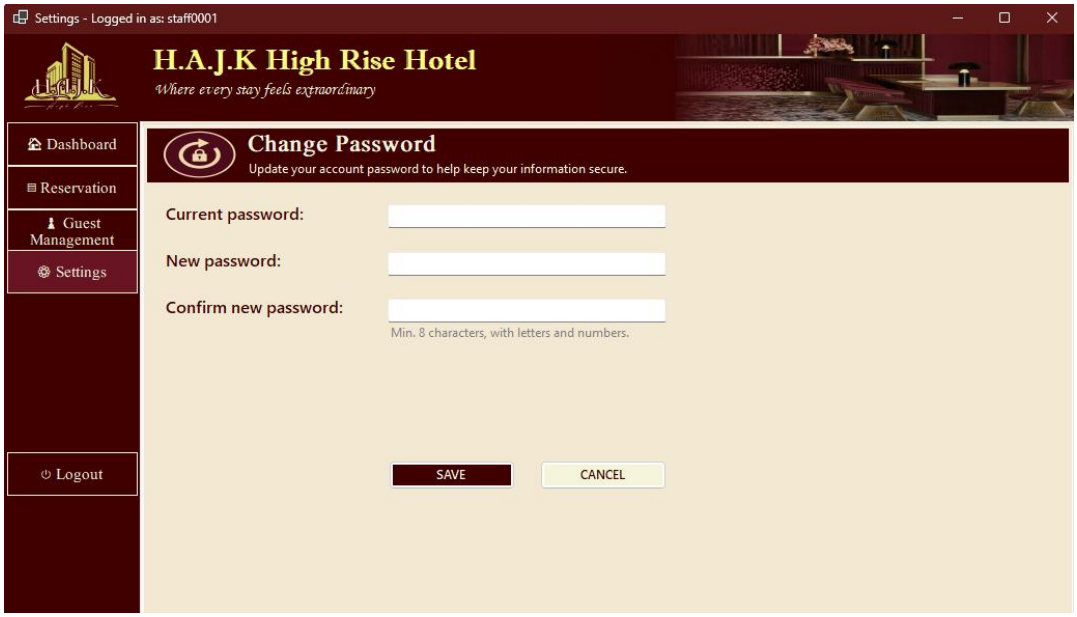


Figure 19: Staff Account Settings – Password Modification Interface

The restricted security settings view accessible from staff accounts to update their personal login credentials. It enforces password complexity standards and verifies current credentials before updating the database records.

Appendix B – Test Case Documentation

This section presents the execution outputs and test validation results for functional, security, and exception-handling scenarios across the H.A.J.K. High-Rise Hotel Reservation System.

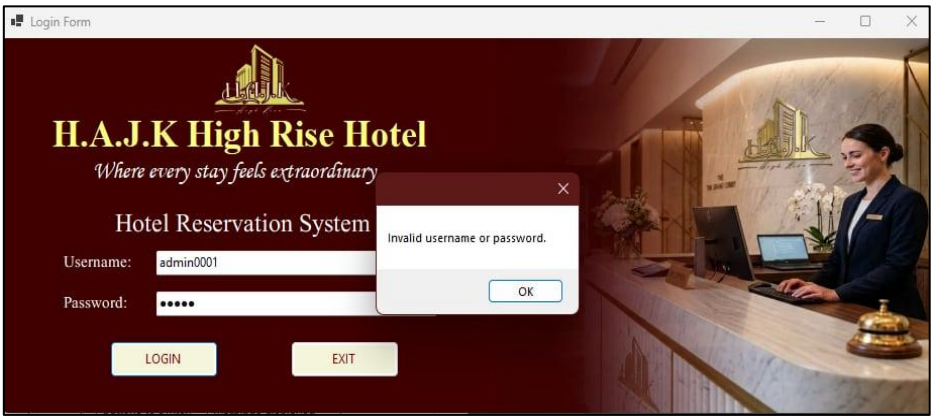


Figure 20: Invalid Login Credentials Error Output

The application correctly detects invalid login credentials and triggers a pop-up error dialog stating "Invalid username or password" to prevent unauthorized access.

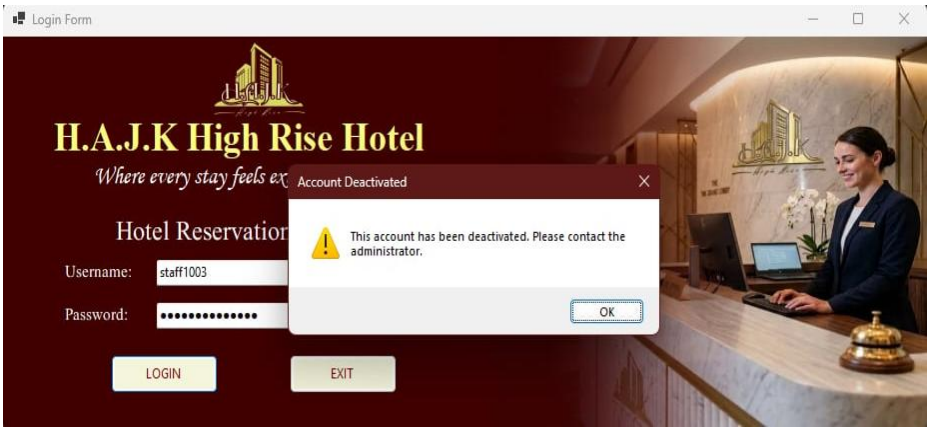


Figure 21: Deactivated Account Login Restriction

Tests system response when an inactive or deactivated staff account attempts to log in. The system blocks authentication and displays a warning dialog advising the user that the account has been deactivated.

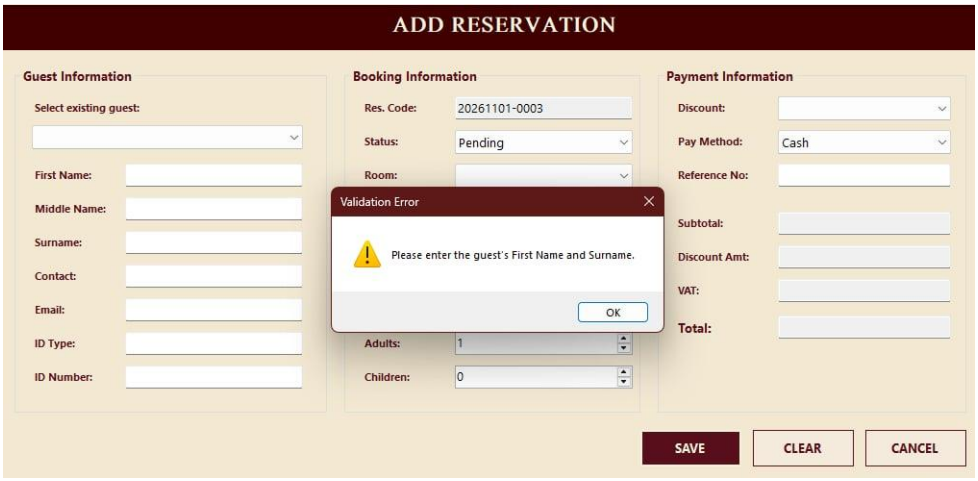


Figure 22: Missing Guest Name Fields in Reservation

Evaluates input validation logic within the reservation module. When mandatory guest identifier fields are left blank during reservation entry, the system halts execution and prompts a validation error requiring the user to enter the guest's First Name and Surname.

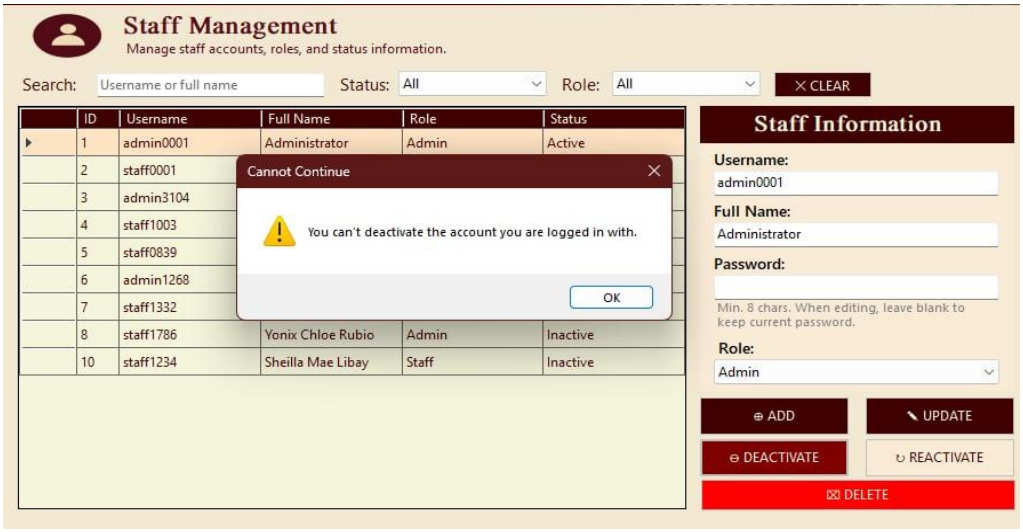


Figure 23: Self-Deactivation Prevention

Tests the safety constraint within the Staff Management module that prevents active administrators from accidentally or maliciously locking themselves out by deactivating their own currently logged-in account.

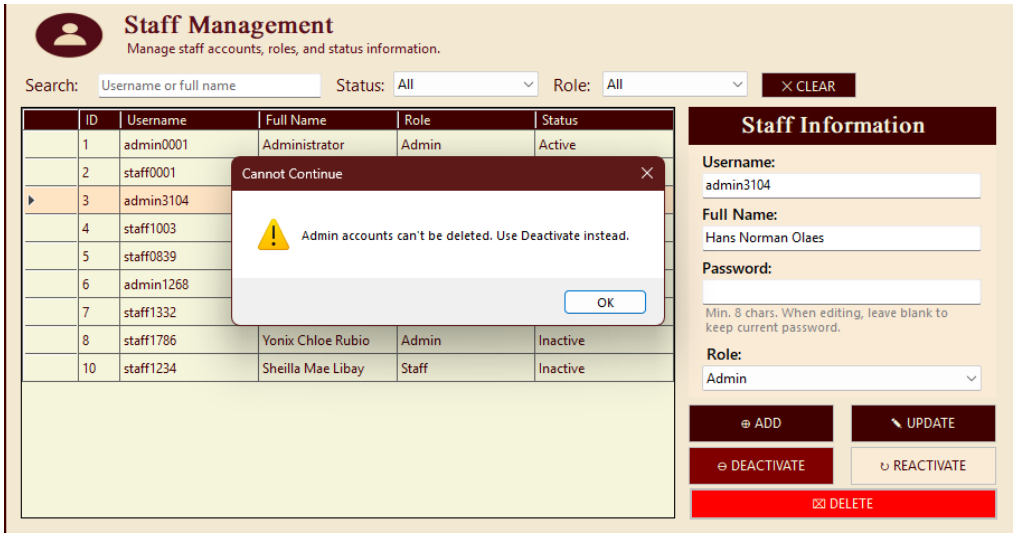


Figure 24: Admin Account Deletion Restriction

Validates system security rules preventing the permanent deletion of administrator records from the database table, enforcing the use of the deactivation protocol instead.

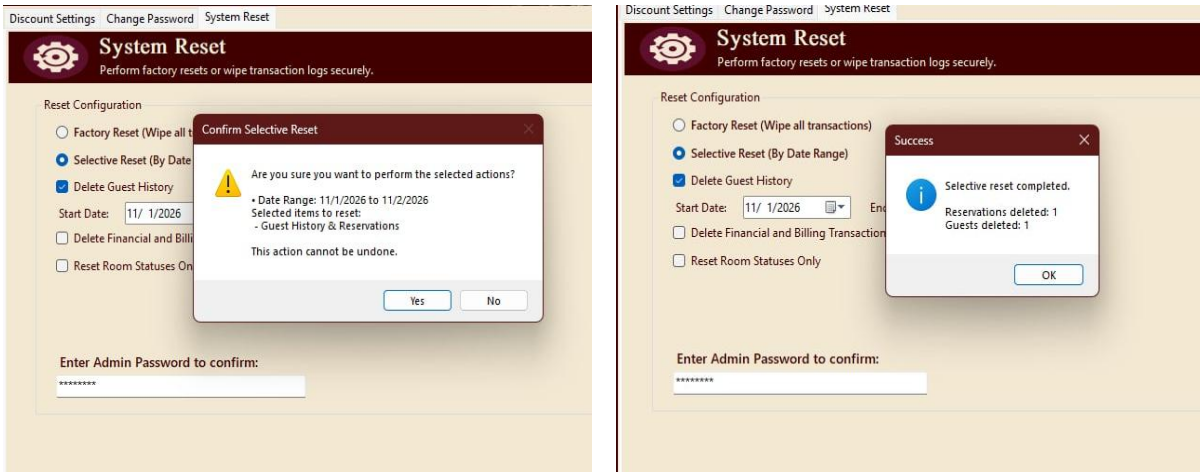


Figure 25: Selective Database Reset Execution Test

Requires admin password re-verification and displays the targeted date range (11/1/2026 to 11/2/2026) before executing irreversible data purges. Success notification displays the post-execution feedback dialog confirming the successful completion of the selective reset, itemizing the exact count of deleted reservations and guest records.

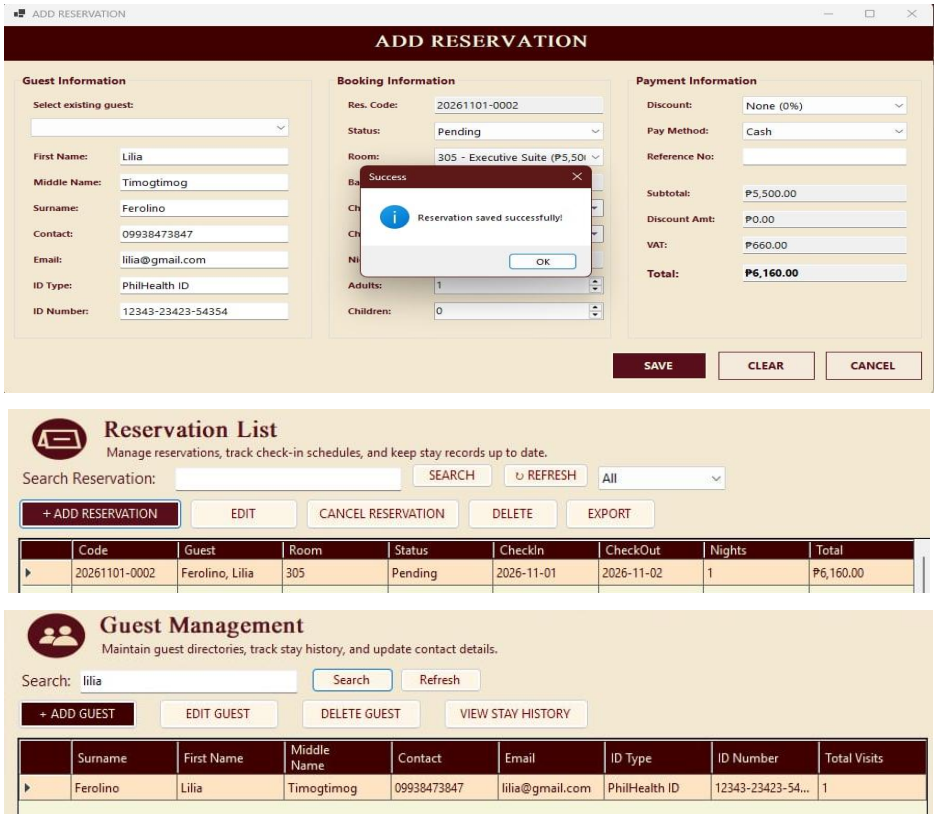


Figure 26: Reservation Process Validation Test

Validates the complete workflow for creating a new booking for new guests in Room 305, confirming accurate financial calculations, accompanied by a success confirmation modal.

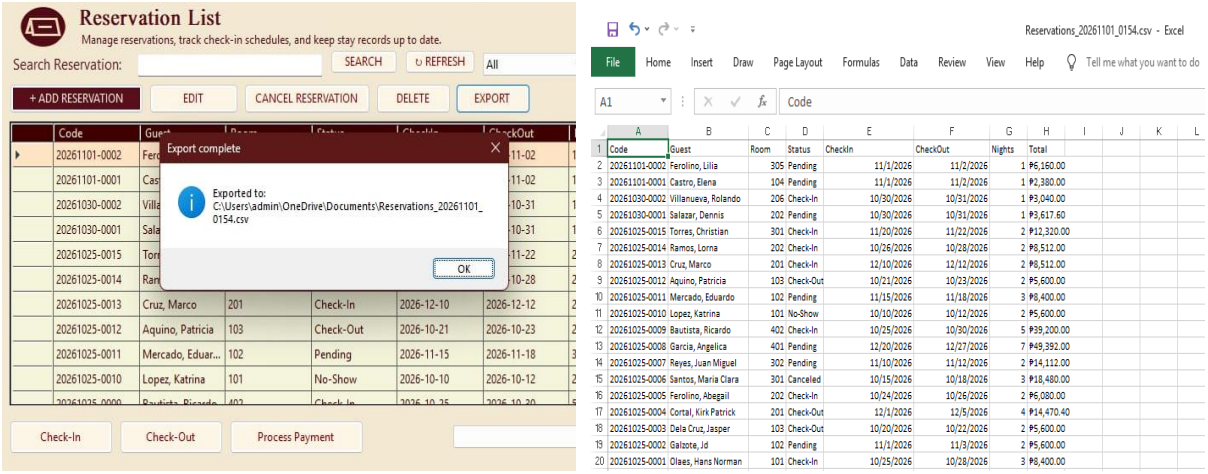


Figure 27: Data Export Functionality & CSV Generation Test Case

Validates the CSV export routine within the Reservation Management module, showing the system's success notification and the generated output file path, and demonstrating complete structural data fidelity upon opening the generated file in Microsoft Excel.

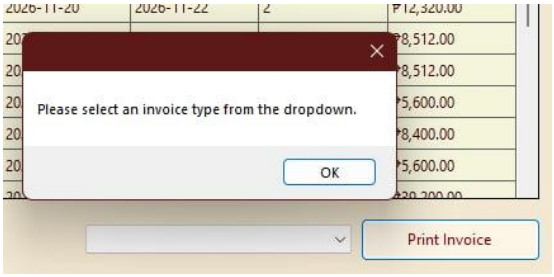


Figure 28: Invoice Generation Validation & Exception-Handling Test Case

Validates input validation logic within the reservation module's invoice printing feature, demonstrating that when a user attempts to trigger the "Print Invoice" action without selecting an invoice format from the accompanying dropdown menu, the system successfully intercepts the request and displays a warning dialog prompting the user to "Please select an invoice type from the dropdown".





H.A.J.K High Rise Hotel

Annual Report for 2026 | Total Revenue: ₱82,390.40

Payment Date	Reservation Code	Guest Name	Amount	Payment Method
2026-10-30 22:12	20261025-0004	Cortal, Kirk Patrick	₱14,470.40	Cash
2026-10-30 20:58	20261030-0002	Villanueva, Rolando	₱3,040.00	Cash
2026-10-30 20:45	20261025-0001	Olaes, Hans Norman	₱8,400.00	Cash
2026-10-30 20:45	20261025-0003	Dela Cruz, Jasper	₱5,600.00	GCash
2026-10-30 20:45	20261025-0005	Ferolino, Abegail	₱6,080.00	Card
2026-10-30 20:45	20261025-0009	Bautista, Ricardo	₱39,200.00	Bank Transfer
2026-10-30 20:45	20261025-0012	Aquino, Patricia	₱5,600.00	Cash

Figure 29: Financial Report PDF Export Test Case

Validates the financial reporting export routine within the Reports module, capturing the system success notification verifying the generated output file path and demonstrating professional document layout fidelity upon opening the exported PDF.

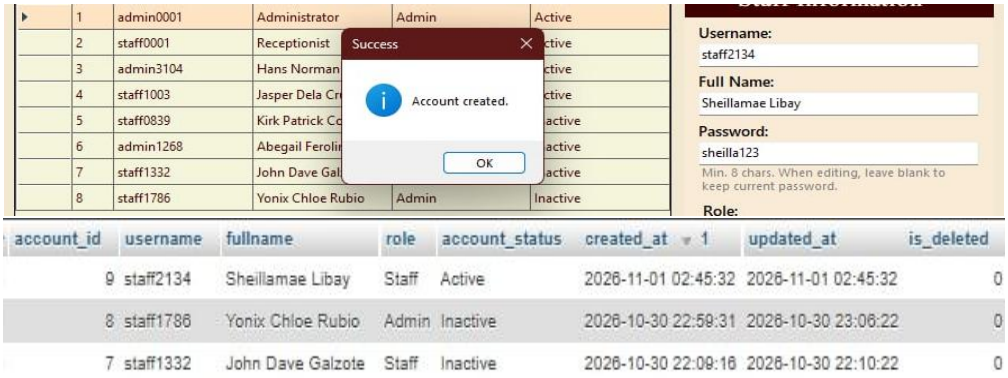


Figure 30: Staff Account Creation & Database Persistence

Validates the staff account creation routine within the administrative Staff Management module, capturing the system success confirmation dialog ("Account created") upon successfully registering a new staff credential and demonstrating complete back-end relational database persistence upon inspecting the underlying MySQL table, where the new record is accurately stored in the database.

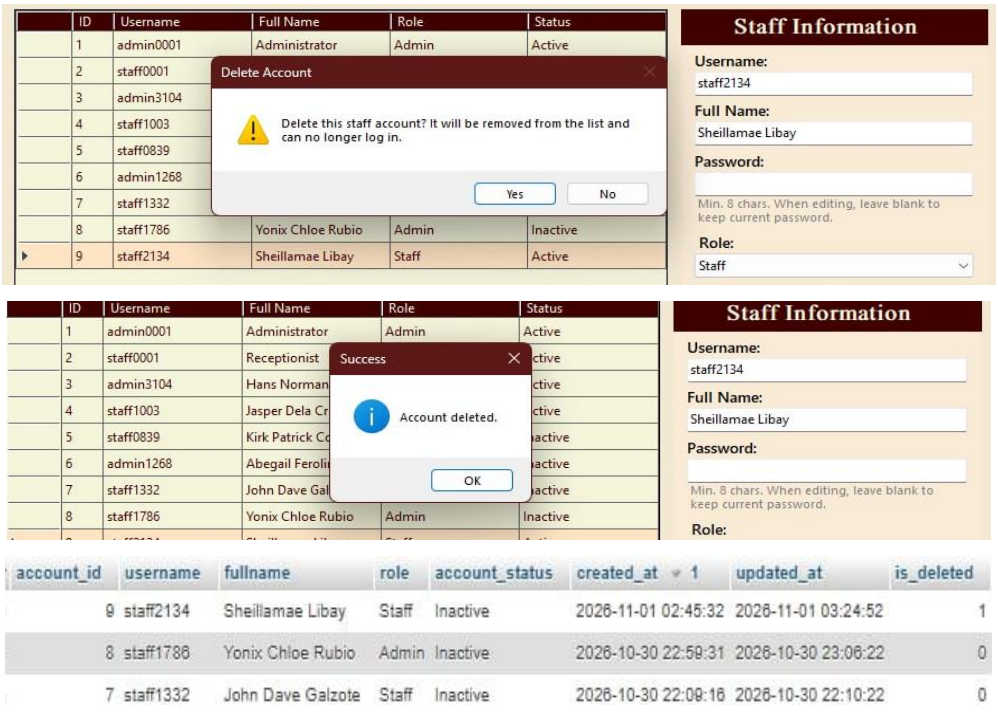


Figure 31: Account Soft Deletion & Database Persistence

Validates the staff account deletion workflow within the administrative Staff Management module, capturing the initial confirmation prompt ensuring secure action verification, displaying the success feedback dialog confirming the removal of the targeted staff record, and confirming back-end relational database integrity under MySQL inspection.

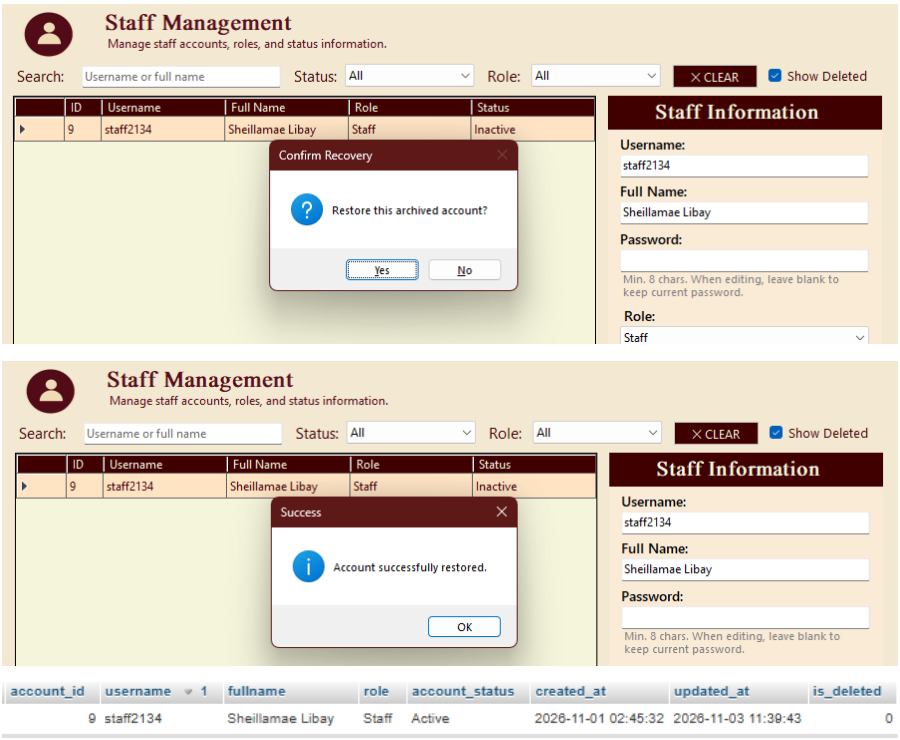


Figure 32: Staff Account Recovery Workflow

Displays the confirmation dialog prompting the user to restore the archived account, and shows the success message confirming that the account has been successfully restored.

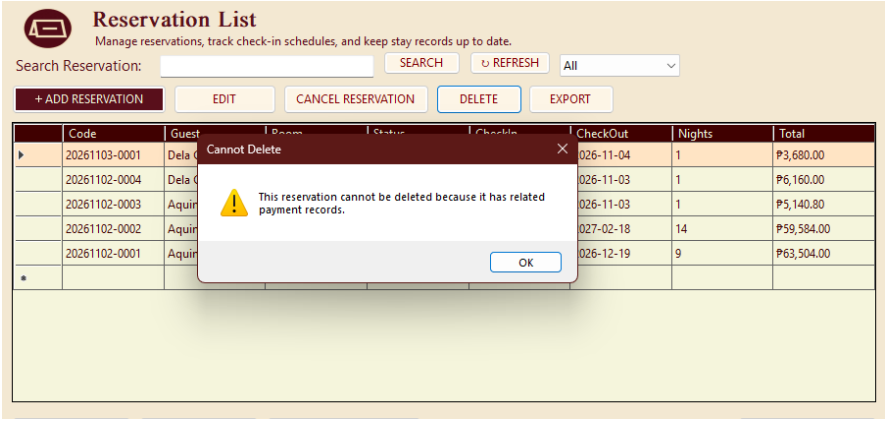


Figure 33: Reservation Deletion Prevention

A user interface view of the Reservation List displaying a dependency safeguard pop-up, which prevents the deletion of any reservation that has associated payment records.

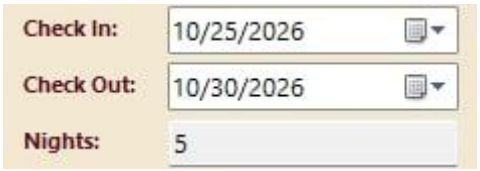


Figure 34: Automatic Nights Duration Count Calculation

Validates the dynamic date-calculation logic within the booking module, demonstrating that when a guest specifies a Check-In date of 10/25/2026 and a Check-Out date of 10/30/2026, the system automatically computes and displays the correct duration of 5 nights.

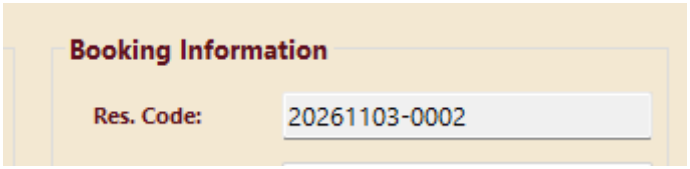


Figure 35: Auto-Generated Reservation Identifier

Illustrates the automatically generated reservation code formatted with the current booking date and daily sequence counter.



Figure 36: Invalid Date Sequence Validation Test Case

Evaluates the date-range validation logic in the booking form, capturing the error handling response when a Check-Out date is set prior to the Check-In date, showing a validation warning dialog stating "Check-out date must be after check-in date".



Figure 37: Standard Reservation Calculation Test

Validates real-time financial computation during booking entry for a standard room across a 2-night stay with 2 adults, verifying that the system accurately calculates the subtotal, applies the standard 12% VAT, and outputs the correct total amount due.

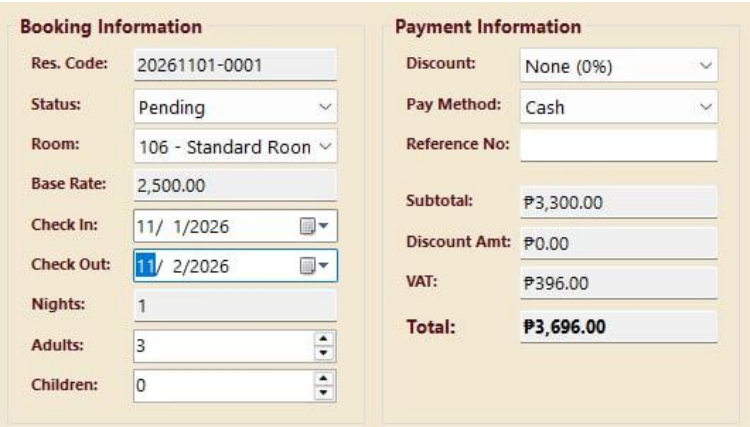


Figure 38: Extra Adult Occupancy Calculation

Tests dynamic pricing adjustments based on occupant count, showing that increasing the number of adults to 3, which is greater than the standard capacity (2) for a 1-night stay recalculates the subtotal to ₱3,300.00 and the VAT to ₱396.00 for a total of ₱3,696.00.

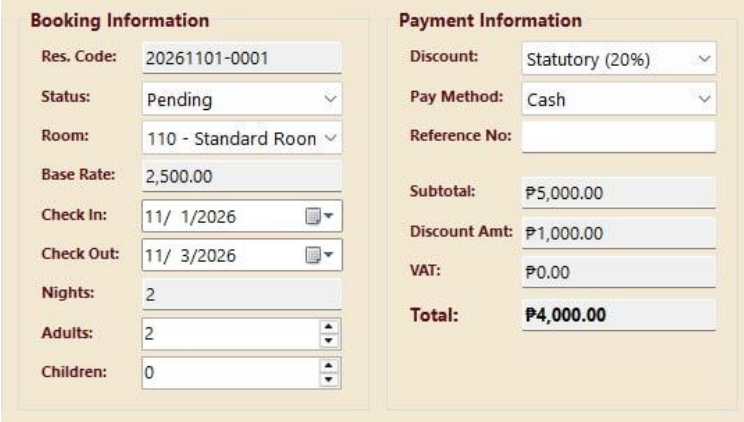


Figure 39: Statutory Discount Application Test Case

Validates the promotional discount processing logic, capturing the application of a Statutory (20%) discount on a 2-night stay in Room 110, where a subtotal of ₱ 5,000.00 yields a discount amount of ₱1,000.00 with zero VAT, resulting in a discounted total of ₱4,000.00.

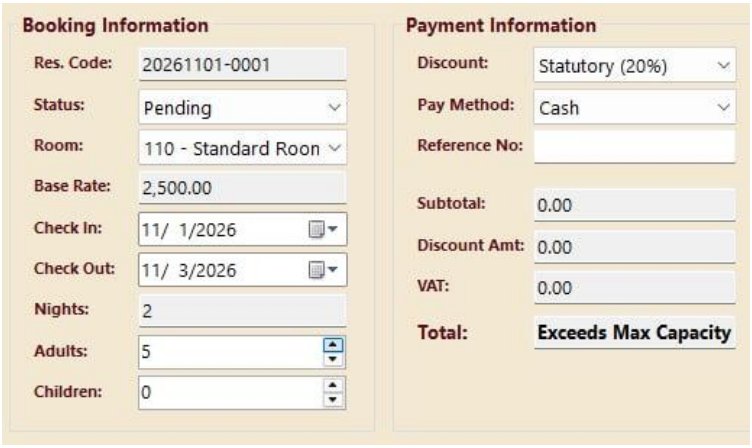


Figure 40: Room Capacity Exceeded Validation Test

Tests system capacity constraints by evaluating booking entry behavior when the number of adults (5) exceeds the maximum allowed limit for the selected room type, causing the system to halt calculation and display an "Exceeds Max Capacity" warning message in the total field.



Figure 41: Lean Season Discount & VAT Computation

Validates seasonal pricing rules by applying a Lean Season (15%) discount to a 2-night booking in Room 101, demonstrating correct arithmetic where a ₱5,000.00 subtotal applies a ₱750.00 discount and a ₱510.00 VAT, totaling ₱4,760.00.